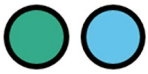
CHAPTER 13

Applying the ADM



This chapter is part of the GREEN and BLUE reading tours. See Figure 1-1 in Chapter 1, Section 1.3, for alternative reading tours.

The Architecture Development Method describes a structured approach to achieving a sound Enterprise Architecture in a variety of situations and scenarios. The step-by-step approach – not to be confused with a waterfall approach – allows architects to address each sub-area of an architecture challenge in a very focused manner.

The Architecture Development Method can be used in combination with a variety of architectural styles, and the iterative nature of the method makes it possible to respond effectively to different situations. This makes the Architecture Development Method ideal for supporting architecture development. Together with the various applications of the method, the Architecture Development Method provides a solid approach for addressing any architectural issue.

© Eric Jager 2025
E. Jager, *Mastering the TOGAF® Standard*, https://doi.org/10.1007/979-8-8688-1814-1_13

13.1. Various Architecture Development Method Approaches

The Architecture Development Method can be adapted to a number of different scenarios and situations. Its tailorability makes it an easy method to adopt when dealing with different process styles or even specialized architectures such as Security Architecture. Because of its adaptability, the Architecture Development Method can be applied to almost any situation for any organization.

There are three specific scenarios that require further explanation:

- **Using the TOGAF Standard with Different Architectural Styles:** This section explains how the TOGAF Standard can be used and adapted to different architectural styles.
- **Applying Iteration to the Architecture Development Method:** This section discusses the concept of iteration and provides an overview of how it can be applied to the Architecture Development Method.
- **Architecture Partitioning:** This section shows how partitions can be used to structure and simplify architecture development.

There are, of course, many other possible applications of the Architecture Development Method. However, the situations or specific applications just mentioned generally cover most of them.

13.2. Using the TOGAF Framework with Different Architecture Styles

The TOGAF Standard is an exceptionally versatile framework, carefully crafted to accommodate a wide variety of architectural styles (e.g., Service-Oriented Architectures, Microservices Architectures, or Zero-Trust Architectures). These styles vary in focus, form, techniques, materials, subject matter, and historical context. As a comprehensive framework, the TOGAF Standard is designed to be adaptable to different environments, allowing organizations to tailor it to their unique architectural needs.

Within an organization's architectural landscape, it's common to encounter architectural work developed in multiple styles. The TOGAF Standard ensures that the requirements of each stakeholder are appropriately addressed, taking into account the perspectives of others and the Baseline Architecture.

When using the TOGAF Standard to support a particular architectural style, architects must consider the unique characteristics of that style. The first step is to identify these unique characteristics. Next, architects determine how to address these characteristics within the framework. This typically involves adapting models, views, and tools, rather than making significant changes to the TOGAF Framework.

During phases B, C, and D, architects are expected to select relevant architectural resources – including models, viewpoints, and tools – to accurately describe the architecture domains and ensure that stakeholder concerns are addressed. Depending on the unique features, different architectural styles may introduce new elements, emphasize existing ones, adjust notations, and focus attention on specific stakeholders or their concerns. Addressing these unique features often requires extending the Enterprise Metamodel and Content Framework, using specific notations or modeling techniques, and identifying appropriate viewpoints.

For example, a predominant architectural style may cause architects to revisit the Preliminary Phase to modify the Architecture Capability or to address special features within a single Architecture Development Method cycle. Style-specific reference models and maturity models are commonly used tools to assist architects in this endeavor.

Throughout its life, the TOGAF Framework has evolved to encompass different architectural styles, address key challenges faced by practitioners, and demonstrate its adaptability within defined contexts.

13.3. Applying Iteration to the Architecture Development Method

The Architecture Development Method of the TOGAF Standard emphasizes flexibility and adaptability in the architecture development process. Iteration plays a key role in ensuring that the architecture evolves in response to changing requirements, feedback, and context. The Architecture Development Method is iterative in nature, meaning that it can be revisited as needed. Architects continually evaluate and refine the architecture as they move through the phases, ensuring that it remains relevant and effective. The end result is a comprehensive and well-aligned Enterprise Architecture that supports the organization's business goals.

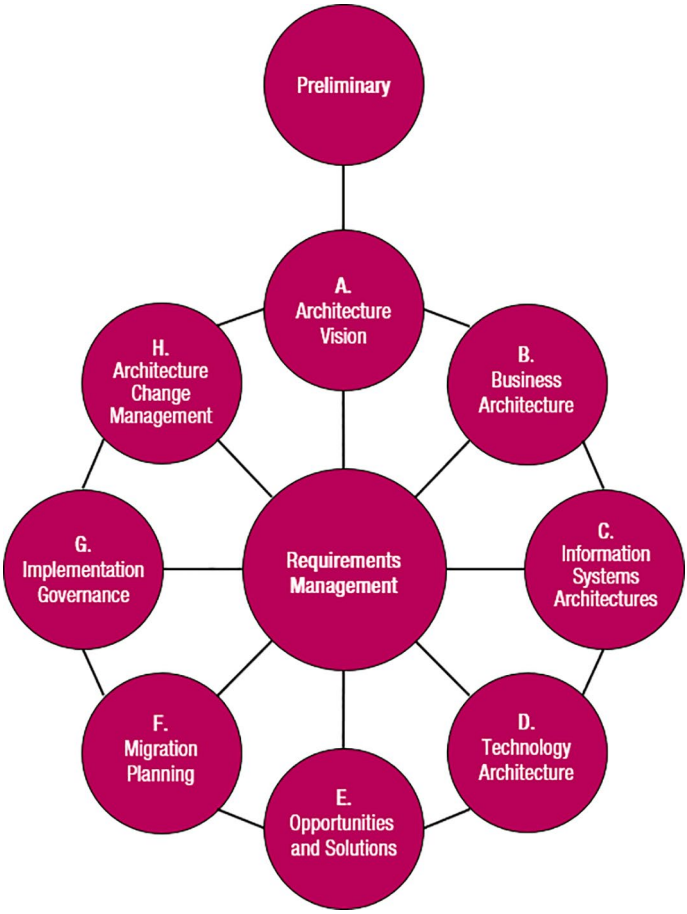


Figure 13-1. *The Architecture Development Method graphic*

It is still quite common to misunderstand the concept of the Architecture Development Method, especially regarding its iterative nature. The Architecture Development Method graphic (Figure 13-1) is frequently misinterpreted as a linear, waterfall-style process. In reality, it is a logical method that groups key activities to clarify their relationships and information flow. The well-known graphic is a stylized representation showing these information flows, and it is *not a representation of an activity sequence.*

> The Architecture Development Method should not be understood as a process model [18].

It's crucial to recognize that whenever the Enterprise Architecture team engages in any activity within the Architecture Development Method's scope, they are executing a phase and contributing to the Enterprise Architecture Landscape. For instance, when an architect works on roadmap development, they are engaging in Phase E (Opportunities and Solutions). This requires consuming mandatory inputs and producing mandatory outputs, a principle applicable to all Architecture Development Method phases.

13.3.1. Different Ways to Apply Iteration

According to the TOGAF Standard, there are three ways to apply iteration to the Architecture Development Method:

- Iteration across the whole ADM cycle
- Iteration within an ADM cycle
- Iteration to manage the Architecture Capability

These approaches reflect different levels and ways in which the Architecture Development Method can be tailored based on an organization's needs or specific architectural context.

Iteration Across the Whole Architecture Development Method Cycle: This type involves executing *the entire Architecture Development Method cycle multiple times* for different levels of architecture or across different scope definitions (e.g., organization-level versus project-level architectures).

Applying iteration to the whole Architecture Development Method cycle is often used when a comprehensive Architecture Landscape is developed. This can be done in layers or when working in an agile or incremental development approach, where the entire process is applied iteratively to build upon previous cycles.

After completing an Architecture Development Method cycle for a specific architectural layer (e.g., Business Architecture), the process is applied again for other layers (Information Systems Architectures and/or Technology Architecture).

The key difference between applying iteration to the whole Architecture Development Method cycle and other ways of applying iteration is that this method of iteration is typically used to address multiple layers or versions of an architecture, such as transitioning from a high-level Enterprise Architecture to more detailed architectures.

Iteration Within an Architecture Development Method Cycle: In this approach, iteration occurs *between two or more phases* of the Architecture Development Method cycle.

This type is used for moving back and forth between different phases, revisiting previous phases as new information or insights emerge.

Applying iteration between two or more phases of the Architecture Development Method cycle is helpful when these phases have interdependencies or when new insights from one phase necessitate reworking a previous phase. Think, for example, about managing the interrelationship between Business Architecture, Information Systems Architectures, and Technology Architecture. Also, applying iteration within an Architecture Development Method cycle can be used to converge on a detailed Target Architecture when higher-level architecture does not exist to provide context and constraint.

The three most common groups of iteration within an Architecture Development Method cycle are called *Architecture Development*, *Transition Planning*, and *Architecture Governance*. Figure 13-2 illustrates these three iteration groups.

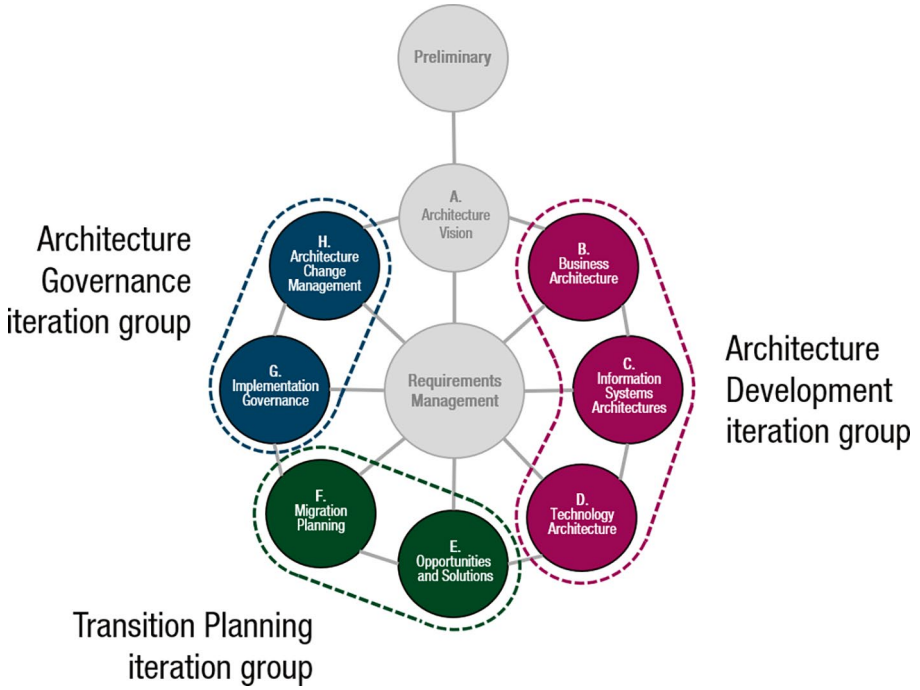


Figure 13-2. *Three most common iteration groups*

The first of the three iteration groups is called the *Architecture Development iteration group*. This particular iteration is commonly used for developing the Baseline and Target Architectures during phases B, C, and D.

The second scenario where iteration occurs between two or more phases is to arrive at an executable Architecture Roadmap or Implementation and Migration Plan. This type of iteration is triggered when the implementation details and scope of change demand a change or reprioritization of stakeholder requirements. Also, there may be changes in business strategy, new requirements may emerge, or issues may arise that require re-evaluation of earlier decisions. A common name for this particular iteration group is the *Transition Planning iteration group*.

The third situation that calls for applying iteration within an Architecture Development Method cycle is to support governance of change activities when moving toward a Target Architecture. This specific iteration group is called the *Architecture Governance iteration group*.

The key difference between applying iteration to two or more phases and other ways of applying iteration is that this form of iteration *involves moving backward or looping between distinct phases* to refine the architecture as it evolves across the lifecycle. This type of iteration is focused on deepening the activities within one or more phases without affecting the overall flow of the Architecture Development Method.

Consider the following scenario. An organization has two goals: implement new regulatory requirements and efficiently use human resources. In order to achieve both goals, certain architectural things need to be accomplished. In this scenario, it is assumed that there are two initiatives (Initiative A and Initiative B), each associated with a specific goal, that are needed to achieve the described goals (see Figure 13-3).

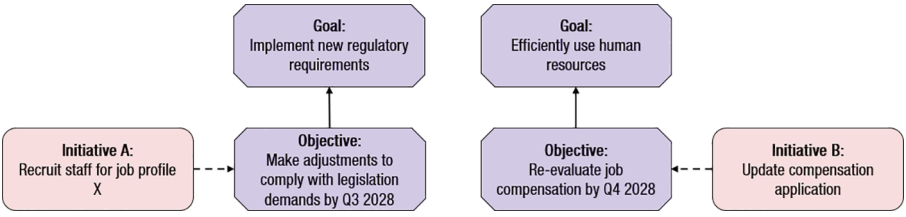


Figure 13-3. *Goals and related initiatives*

Each initiative requires its own approach and adaptation to the organization's setup.

Initiative A indicates that new employees with the required profile must be hired. As a result, HR will need to invest time and effort in finding and hiring the right people. This will require, among other things, the Inflow Support capability.

Initiative B involves updating the application responsible for determining the appropriate compensation for the job profiles. The job profiles need to be re-evaluated. This will require an investment in new software. The amount of the investment is equal to the investment that HR spends on recruiting (in terms of internal staff hours).

During phases B and C of the Architecture Development Method, the two possible scenarios described above are developed. Both scenarios are presented to the CIO during Phase F (Migration Planning). Since the available budget can only be spent once, the CIO must choose between the two proposed scenarios. The CIO uses the Architecture Alternatives and Trade-Offs method to determine the final direction (see Figure 13-4).

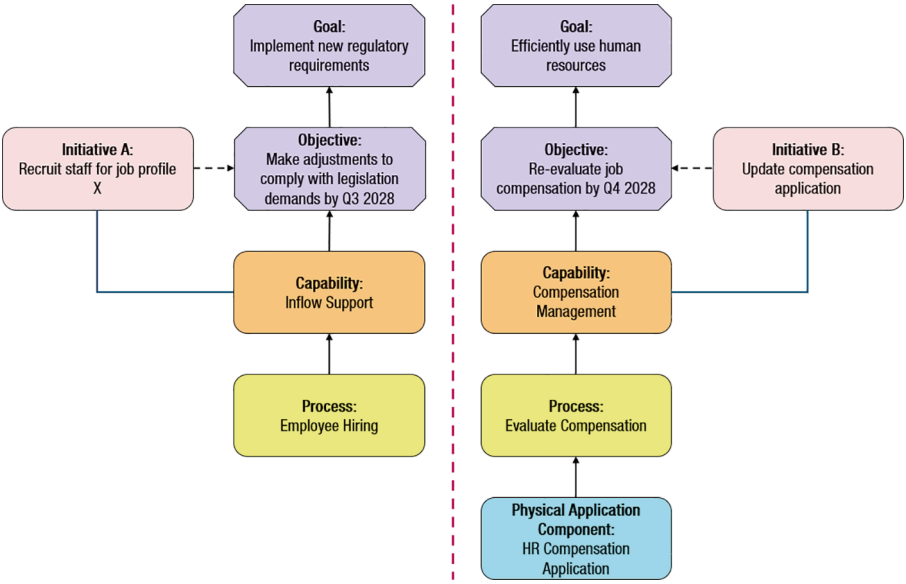


Figure 13-4. *Two possible scenarios created when applying the Architecture Alternatives and Trade-Offs method*

For the selected scenario, another *Architecture Development iteration cycle* is initiated to further refine the organizational implications.

Iteration to Manage the Architecture Capability: This type involves executing *the Preliminary Phase and Phase A* in order to adjust (parts of) the Architecture Capability.

The third type of iteration concerns itself solely with the Architecture Capability and is therefore fittingly called the *Architecture Capability iteration group* (see Figure 13-5).

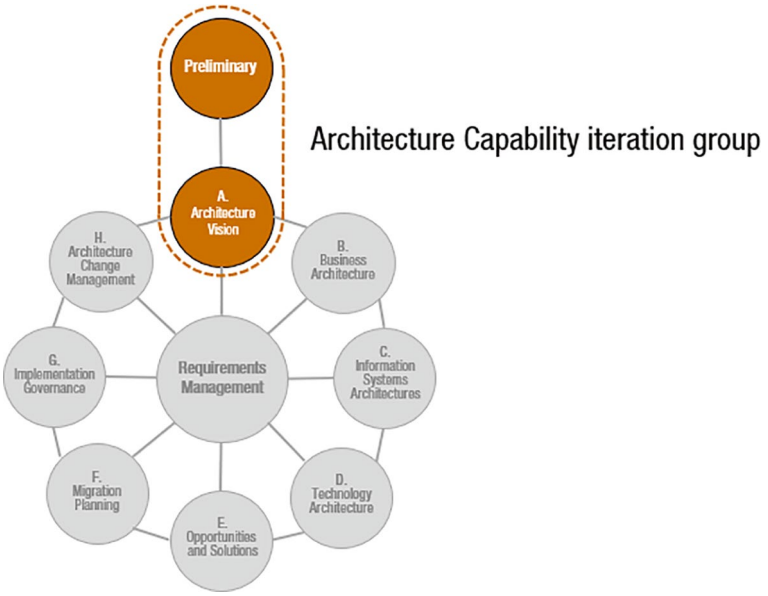


Figure 13-5. *Architecture Capability iteration group*

The iteration of the Preliminary Phase and Phase A (Architecture Vision) may be triggered by projects wanting to (re-)establish aspects of the Architecture Capability identified in Phase A (Architecture Vision). A second example is when projects require a new iteration of the Preliminary Phase to adjust the organization's Architecture Capability. This need may arise from the identification of new or changed requirements for the Architecture Capability as a result of a change request in Phase H (Architecture Change Management).

In short, applying iteration to the Preliminary Phase and Phase A (Architecture Vision) is primarily used to process changes to the Architecture Capability. Iteration between Architecture Development Method phases means revisiting earlier phases based on new insights or changes and focuses on the development of architecture. Finally, applying iteration across the whole Architecture Development Method cycle requires the entire cycle to be repeated iteratively. This is often applied to different layers of the architecture. It is used to develop a comprehensive Architecture Landscape.

13.3.2. How to Apply Iteration

The TOGAF Standard indicates that for a typical architecture approach, it is prudent to apply the iteration cycles of the Architecture Development Method. There are four iteration groups related to architecture activities. Each group has a specific purpose. Figure 13-6 illustrates the iteration groups:

Architecture Capability: These iterations of the Architecture Development Method are designed to support the establishment and continued growth of the Architecture Capability. This means mobilizing the organization to proceed with the implementation of architecture activities. A motivator for this may be that the organization in question wants to start working with architecture or that a need for an architecture approach has arisen. During this iteration, the focus is on establishing the architecture approach, defining principles, and determining the scope of the architecture. Developing an architecture vision and establishing architecture governance are also part of this iteration group. The phases of the Architecture Development Method used are the Preliminary Phase and Phase A (Architecture Vision).

Architecture Development: Iterations whose purpose is to develop architecture content. The phases associated with architecture development are phases B–F. Going through these phases of the Architecture Development Methodology creates a cohesive architecture. Typically, to shape the Baseline Architecture, the iteration will focus on Phase B (Business Architecture), Phase C (Information Systems Architectures), and Phase D (Technology Architecture). The stakeholder reviews related to the architecture are often broader in scope during this iteration. As the iterations move toward the Target Architecture, Phase E (Opportunities and Solutions) and Phase F (Migration Planning) in particular will ensure the feasibility of the architecture to be implemented.

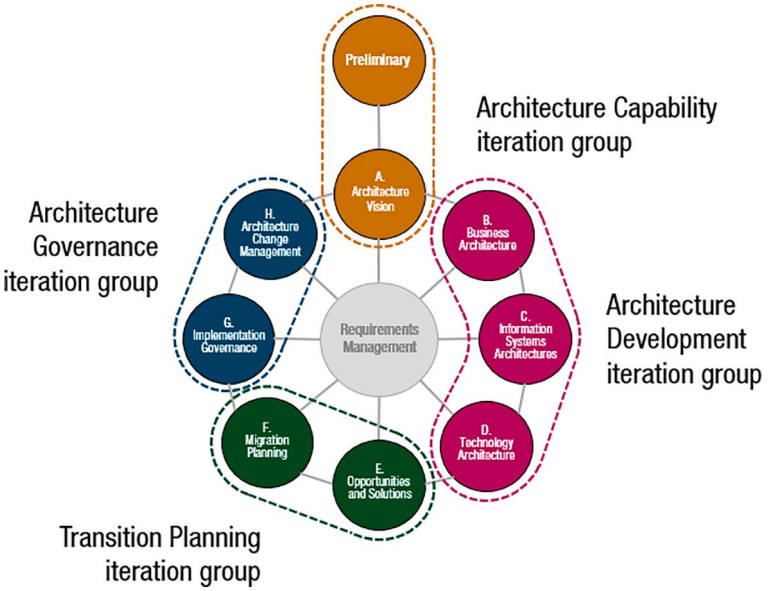


Figure 13-6. *Iteration groups of the ADM*

Transition Planning: Iterations in this group support the creation of roadmaps for change initiatives related to the architecture.

The purpose of the iterations is to properly plan the transition from the Baseline to the Target Architecture. Phase E (Opportunities and Solutions) and Phase F (Migration Planning) are the Architecture Development Method phases used within this group of iterations.

Architecture Governance: All iterations that take place with the goal of establishing and monitoring architecture governance. The iterations of the Architecture Development Method emphasize Phase G (Implementation Governance) and Phase H (Architecture Change Management).

The four iteration groups and their corresponding Architecture Development Method phases are listed in Table 13-1. The table also shows the possible iteration cycles for each iteration group.

Table 13-1. *Activity by iteration cycle*

Iteration Group		**Architecture Development**			**Transition Planning**		**Architecture Governance**	
Architecture Development Method Phase		**Iteration 1**	**Iteration 2**	**Iteration *n***	**Iteration 1**	**Iteration *n***	**Iteration 1**	**Iteration *n***
Preliminary								
Architecture Vision								
Business Architecture	Baseline	X		x				
	Target		x	x				
Application Architecture	Baseline	X		x				
	Target		x	x				
Data Architecture	Baseline	X		x				
	Target		x	x				
Technology Architecture	Baseline	X		x				
	Target		x	x				
Opportunities and Solutions					x	x		
Migration Planning					x	x		
Implementation Governance							x	x
Change Management							x	x

Establishing the architecture approach, defining the principles, developing an architecture vision, and defining the scope do not need to be repeated for every type of architecture work. For this reason, the corresponding phases of the Architecture Development Method (Preliminary Phase and Phase A (Architecture Vision)) in Table 13-1 are not a regular part of the more common iterations (shown in Figure 13-7) that aim to develop the architecture, plan its transition, and monitor its implementation.



Figure 13-7. *Iteration cycles Architecture Development, Transition Planning, and Architecture Governance*

The first iteration cycle of the Architecture Development iteration group has architecture development as its goal. This cycle can be initiated in one of two ways. The cycle can be started with a focus on the *Baseline Architecture* or with a focus on the *Target Architecture*.

Baseline First: The first iteration within this cycle is to create a Baseline Architecture (see Figure 13-8). The phases to be completed are the Business Architecture, Information Systems Architectures, and Technology Architecture, focusing on the definition of the baseline. Opportunities, solutions, and migration plans are also considered to drive out the focus for change and test feasibility.

The second iteration is aimed at defining the Target Architecture and identifying the gaps between the Baseline and the Target Architecture. This iteration again consists of going through the Business Architecture, Information Systems Architectures, and Technology Architecture phases of the Architecture Development Method. Only this time, it focuses on defining the Target Architecture and analyzing the gaps against the Baseline Architecture.

Opportunities, solutions, and migration plans are included in the feasibility review process. All subsequent iterations are aimed at correcting and refining the Baseline and Target Architecture and associated gaps to achieve an outcome that is beneficial, feasible, and viable.

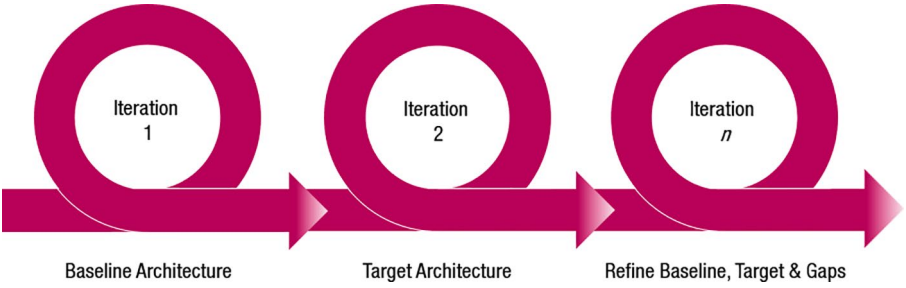


Figure 13-8. *Iteration cycles of the Architecture Development activity*

Target First: The second way to apply the first iteration cycle is to develop the Target Architecture as the first step. The approach to the cycle is much the same as starting with the Baseline Architecture, except that the focus is on first mapping the desired architecture and then comparing it to the Baseline Architecture. This is a common starting point in many organizations.

The second iteration cycle of the Architecture Development iteration group deals with transition planning to get from the Baseline to the Target Architecture. The goal of the first iteration within this cycle is to define and agree on a set of improvement opportunities, aligned against a provisional Transition Architecture. The initial iteration of Transition Planning (see

Figure 13-9) seeks to gain buy-in to a portfolio of solution opportunities in the Opportunities and Solutions Phase of the Architecture Development Method. This iteration also delivers a provisional migration plan.

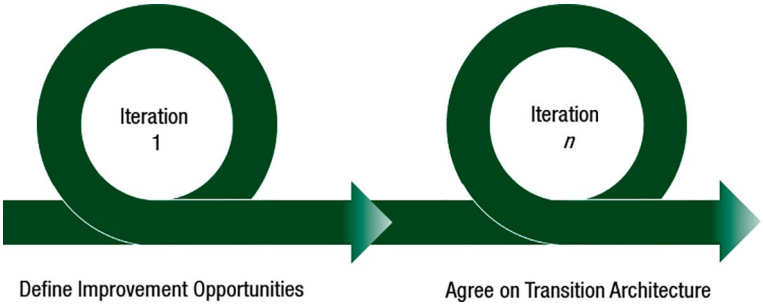


Figure 13-9. *Iteration cycles of the Transition Planning activity*

The second step in this cycle is to agree on the Transition Architecture and to refine the identified improvement opportunities in order to make them fit. This iteration of Transition Planning seeks to refine the migration plan, feeding back issues into the Opportunities and Solutions Phase for refinement.

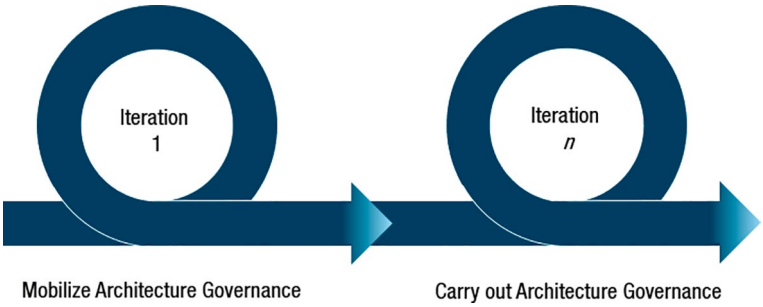


Figure 13-10. *Iteration cycles of the Architecture Governance activity*

Finally, it is also important to pay attention to Architecture Governance (see Figure 13-10). This is done in the third and final iteration cycle. For this topic, the TOGAF Standard suggests two iterations of the Architecture

Development Method, namely, to *mobilize architecture governance* and change management processes during the first iteration and to *carry out architecture governance* and change control during the second.

The initial Architecture Governance iteration establishes a process for governance of change and also puts in place the appropriate people, processes, and technology to support managed access to and change of the defined architecture.

Subsequent iterations of the Architecture Governance cycle focus on periodic reviews of change initiatives to resolve issues and ensure compliance. Results of a change request may trigger another phase to be revisited, for example, feeding back a new requirement to the Preliminary Phase to improve the Architecture Capability or a new requirement for the architecture into the Architecture Development phases.

Many architects often perceive the Architecture Development Method as a linear process model. However, this interpretation is a misconception. The traditional Architecture Development Method diagram is *not an activity diagram*; rather, it is a logical method that clusters essential activity steps to link actions and information flow, producing specific outputs.

Each time an architect performs activities within the scope of the Architecture Development Method, they contribute to the development of the Enterprise Architecture Landscape through iteration. The phase being performed corresponds to the appropriate domain. Attempting to impose a one-pass linear sequence on the Architecture Development Method can result in convoluted, looping phase diagrams. Instead, consider the steps as a checklist that allows for flexibility and adaptability in the architecture process.

By applying iteration, the Architecture Development Method provides a comprehensive and flexible approach to Enterprise Architecture development that can adapt to changes in scope, context, and requirements. This enhances the likelihood of creating an architecture that evolves with the organization's needs over time.



Further Reading

The TOGAF Standard Reference Cards in the TOGAF Library provide a concise overview of the use of iteration in implementing the Architecture Development Method:

https://publications.opengroup.org/togaf-library

The Series Guide, "A Practitioners' Approach to Developing Enterprise Architecture Following the TOGAF ADM", document number G186, provides additional practical information on applying the Architecture Development Method with an eye toward iteration.

13.4. Architecture Partitioning

Architecture partitioning refers to the practice of dividing an Enterprise Architecture into smaller, manageable parts to improve governance, scalability, and efficiency in the architecture development process.

Organizations are sometimes large and complex, making it difficult to develop and govern architecture as a single, monolithic structure. Architecture partitioning allows breaking the architecture into smaller, more manageable segments, improving focus and clarity.

13.4.1. Why Use Architecture Partitioning

One of the primary reasons for implementing architecture partitioning is the fact that different areas of the architecture (Business, Data, Application, and Technology) may have different stakeholders, policies, and compliance requirements. Partitioning ensures that governance can be effectively applied to specific areas without imposing unnecessary constraints on the entire organization.

In addition, architecture partitioning allows different teams to work in parallel on different architecture segments or projects. In fact, it prevents bottlenecks by ensuring that architecture efforts are distributed rather than centralized in a single group. Not only can multiple teams work on pieces of the architecture, but those pieces can be created in different timeframes. Architecture development does not have to happen all at once; it can evolve incrementally. The Architecture Development Methodology can be applied iteratively to different partitions, allowing for incremental improvements.

In layman's terms, architecture partitioning is the process of dividing the architecture work among multiple teams. Along with dividing the workload, definitions are provided so that each team understands what they are supposed to create in terms of the architecture. This also prevents the creation of architecture that doesn't integrate with other parts of the architecture.

The relationship between architecture partitioning and agile approaches is evident. Both involve breaking down work into manageable units.

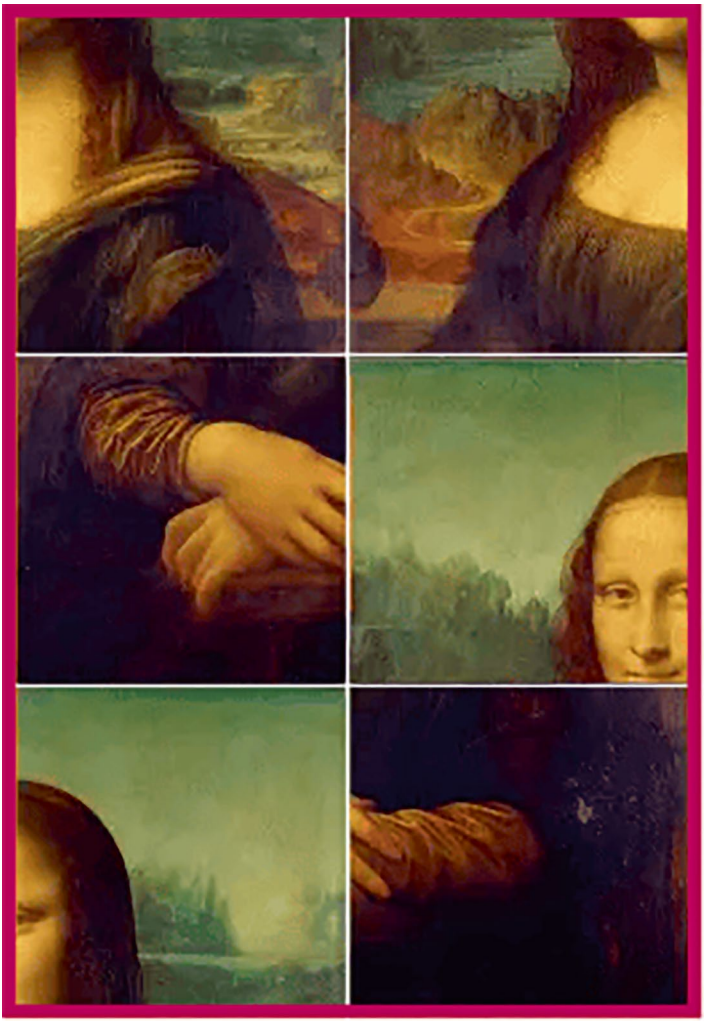


Figure 13-11. *Architecture created without an overall picture*

However, it is crucial to maintain a holistic perspective during this process. Without considering the broader context, there's a risk of developing disjointed architecture components that lack integration and coherence. Figure 13-11 illustrates this undesirable outcome.

13.4.2. Supporting Architecture Partitioning

During the Preliminary Phase, the necessary steps need to be taken in order for the architecture work to support architecture partitioning. The following aspects need to be determined in order for the partitioning to work:

- **Organizational Structure:** This step should determine the teams tasked with the creation of (parts of) the architecture.
- **Responsibilities for the Teams:** It is imperative that a definition of the architecture to be created is agreed upon. At the very least, the definition should include the subject matter areas being covered, the level of detail to use, and what the time periods are during which to deliver the architecture.
- **Relationship Between Architectures:** Needless to say that all teams should work toward an integrated architecture. Any overlap between the various parts of the architecture should be addressed so that reusable building blocks remain.

Once the Preliminary Phase is complete, the teams conducting the architecture should be understood. Each team should have a defined scope, and the relationships between teams and architecture should be understood.

13.5. Summary

The TOGAF Standard is renowned for its adaptability, allowing the Architecture Development Method to be tailored to various scenarios and situations. This flexibility makes it an excellent choice when addressing diverse process styles or specialized architectures.

Three key scenarios include:

- **Using the TOGAF Standard with Different Architectural Styles:** This scenario demonstrates how the TOGAF Standard can be adapted to various architectural styles.
- **Applying Iteration to the Architecture Development Method:** This scenario explores the concept of iteration and provides an overview of its application within the Architecture Development Method.
- **Architecture Partitioning:** This scenario illustrates how partitions can be employed to structure and simplify architecture development.

When leveraging the TOGAF Standard to support a specific architectural style, architects must consider the unique characteristics of that style. The initial step involves identifying these unique traits, followed by determining how to address them. During phases B, C, and D, architects are expected to select relevant architectural resources – including models, viewpoints, and tools – to accurately describe the architecture domains and ensure that stakeholder concerns are addressed. This approach may introduce new elements, emphasize existing ones, adjust notations, and focus attention on specific stakeholders or their concerns.

The concept of the Architecture Development Method is often misunderstood, particularly regarding its iterative nature. Its graphic is frequently misinterpreted as a linear, waterfall-style process, leading to confusion. In reality, it is a logical method that groups key activities to clarify their relationships and information flow. The classic diagram represents this flow, not a sequential activity model.

According to the TOGAF Standard, there are three ways to apply iteration to the Architecture Development Method:

- **Iteration Across the Whole Architecture Development Method Cycle:** This approach involves executing the entire ADM cycle multiple times for different levels of architecture or across varying scope definitions (e.g., organization-level versus project-level architectures).
- **Iteration Within an Architecture Development Method Cycle:** Here, iteration occurs between two or more phases of the ADM cycle, allowing for revisiting previous phases as new information or insights emerge.
- **Iteration to Manage the Architecture Capability:** This type involves executing the Preliminary Phase and Phase A to adjust parts of the Architecture Capability.

Applying iteration cycles within the Architecture Development Method is prudent. There are four iteration groups related to architecture activities:

- **Architecture Capability:** These iterations support the establishment and continued growth of the Architecture Capability.
- **Architecture Development:** Iterations aimed at developing architecture content, typically associated with phases B–F.
- **Transition Planning:** Iterations that support the creation of roadmaps for change initiatives related to the architecture.
- **Architecture Governance:** Iterations focused on establishing and monitoring architecture governance.

The first iteration group, *Architecture Capability*, involves executing the Preliminary Phase and Phase A to adjust parts of the Architecture Capability. The second group, *Architecture Development*, is commonly used for developing architecture content during Phases B through F. The third group, *Transition Planning*, involves iteration between two or more phases with the goal of arriving at an executable Architecture Roadmap or Implementation and Migration Plan. This type of iteration is triggered when implementation details and scope changes necessitate a change or reprioritization of stakeholder requirements. Finally, the fourth group, *Architecture Governance*, supports governance of change activities when progressing toward a Target Architecture.

The key distinction between applying iteration to two or more phases and other forms of iteration is that this approach involves moving backward or looping between distinct phases to refine the architecture as it evolves across the lifecycle.

By applying iteration, the ADM offers a comprehensive and flexible approach to Enterprise Architecture development that can adapt to changes in scope, context, and requirements.

Architecture partitioning is an essential practice in Enterprise Architecture, involving the division of the architecture into smaller, manageable segments.

One primary reason for implementing architecture partitioning is the recognition that different domains within the architecture – such as Business, Data, Application, and Technology – have distinct stakeholders, policies, and compliance requirements. Partitioning allows governance to be effectively tailored to specific areas without imposing unnecessary constraints across the entire organization. Moreover, architecture partitioning facilitates parallel development, enabling various teams to work concurrently on different segments or projects. This approach prevents bottlenecks by distributing architectural efforts, rather than

centralizing them within a single group. Additionally, partitioning supports the incremental evolution of the architecture, allowing teams to develop different components at varying paces, ensuring that architecture development is a continuous process rather than a one-time event.

The relationship between architecture partitioning and agile methodologies is evident. Both involve breaking down work into manageable units. However, it is crucial to maintain a holistic perspective during this process. Without considering the broader context, there's a risk of developing disjointed architecture components that lack integration and coherence.